

El lenguaje HTML

Módulo: Fundamentos de desarrollo Front-End

| AE2: Estructurar el contenido de una página web básica utilizando el lenguaje de etiquetas html5, de acuerdo con un requerimiento específico proporcionado.

Introducción

El lenguaje **HTML (HyperText Markup Language)** es el pilar fundamental del desarrollo web. Con HTML, los desarrolladores pueden crear la estructura y el contenido de las páginas web.

HTML5 es la última versión de HTML y trae consigo nuevas características y mejoras. Proporciona una serie de elementos semánticos que ayudan a describir la estructura del contenido de manera más clara y significativa. Estos elementos incluyen encabezados, secciones, navegación, áreas laterales y pies de página, entre otros.

Aprendizaje esperado

Cuando finalices esta lección podrás:

- Comprender qué es el lenguaje HTML y su importancia en el desarrollo web.
- Entender las características y mejoras introducidas en HTML5 en comparación con las versiones anteriores.
- Aprender sobre los diferentes elementos HTML, sus etiquetas correspondientes y los atributos utilizados para darles propiedades y funcionalidades específicas.
- Analizar la estructura básica de un documento HTML, incluyendo los elementos de una estructura básica de un documento HTML: html, head, body, etc.
- Aprender sobre el elemento <div> y cómo se utiliza como un contenedor genérico para agrupar y organizar otros elementos en bloques.
- Estudiar cómo crear formularios interactivos utilizando los elementos del formulario.

Características de html5

HTML5 es la quinta versión del lenguaje de marcado utilizado para estructurar y presentar contenido en la web. A diferencia de las versiones anteriores, HTML5 presenta una serie de características y mejoras significativas que han revolucionado el desarrollo web.

Características y puntos importantes sobre HTML:

- **Semántica mejorada:** HTML5 introduce nuevos elementos semánticos que permiten describir con mayor precisión la estructura y el significado del contenido. Estos elementos incluyen `<header>`, `<nav>`, `<section>`, `<article>`, `<aside>`, `<footer>` y muchos más.
- **Audio y video nativos:** HTML5 incluye elementos nativos `<audio>` y `<video>` que permiten la reproducción de contenido multimedia sin necesidad de complementos como Flash.
- **Multimedia mejorada:** Además del soporte nativo para audio y video, HTML5 ofrece API para el control y manipulación de medios, como el elemento `<audio>` y `<video>`. Esto incluye la capacidad de controlar la reproducción, obtener metadatos de medios y aplicar efectos y filtros en tiempo real.
- **Gráficos y animaciones:** HTML5 introduce el elemento `<canvas>`, que permite la creación de gráficos, animaciones y visualizaciones interactivas utilizando JavaScript. Además, se ha incorporado el soporte para SVG (Scalable Vector Graphics), lo que permite la utilización de gráficos vectoriales escalables y de alta calidad en las páginas web.
- **Mejoras en formularios:** HTML5 ofrece nuevas opciones de entrada de datos en formularios, como campos de fecha, hora, correo electrónico, número y búsqueda.
- **Almacenamiento local:** HTML5 proporciona API para el almacenamiento local de datos en el navegador, como el almacenamiento en caché, el almacenamiento web y la base de datos de indexación.
- **Compatibilidad con dispositivos móviles:** HTML5 ha sido diseñado pensando en la compatibilidad con dispositivos móviles. Introduce nuevas características y API que facilitan el desarrollo de aplicaciones y sitios web

responsivos, adaptados a diferentes tamaños de pantalla y capacidades táctiles.

- **Mejoras en rendimiento y eficiencia:** HTML5 incluye mejoras en términos de rendimiento y eficiencia, como la carga asíncrona de scripts, la optimización del procesamiento de JavaScript y el uso de almacenamiento en caché.
- **Drag and Drop:** HTML5 facilita la implementación de la funcionalidad de arrastrar y soltar elementos en una página web. Esto permite a los usuarios interactuar de forma intuitiva con los elementos y simplifica la creación de interfaces de usuario interactivas.

Elementos, etiquetas y atributos

En HTML, los **elementos**, **etiquetas** y **atributos** son componentes fundamentales para estructurar y dar formato a un documento web. Aquí tienes una breve descripción de cada uno:

Elementos: En HTML, un **elemento** es una parte o sección de un documento que se delimita mediante **etiquetas**. Los elementos pueden contener texto, otros elementos o ambos.

Las etiquetas HTML están delimitadas **por un inicio y un final** de cada elemento. Lo que se encuentra dentro de la etiqueta (el contenido) es lo que estamos formateado. Toda etiqueta es un juego de pares: **Una etiqueta abre, otra etiqueta cierra.**

```
<etiqueta>  
Contenido  
</etiqueta>
```

Etiquetas: Las etiquetas son símbolos o palabras clave que se utilizan para **marcar** y **delimitar** los elementos en un documento HTML. Las etiquetas están compuestas por un **nombre de etiqueta** encerrado entre los símbolos de **mayor que (<)** y **menor que (>)**.

Atributos: Los atributos proporcionan información adicional sobre un elemento y se especifican dentro de las etiquetas. Los **atributos consisten en un nombre y un valor**, separados por un signo **igual (=)**. Los atributos se utilizan para configurar **propiedades o características** específicas de un elemento. Todas las

etiquetas aceptan atributos. Un atributo es cualquier característica que puede ser diferente entre una etiqueta y la otra.

```
<etiqueta atributo="valor">  
Contenido  
</etiqueta>
```

El valor que tendrá va entre comillas. Una etiqueta puede tener más de un atributo, separados por espacios entre sí. Los atributos sólo van en la etiqueta de apertura.

Sintaxis del código

Al HTML no le importa si las etiquetas las escribís una al lado de la otra o una debajo de la otra. Los “Enter”, espacios (de la barra espaciadora) y tabulaciones no afectan la salida en el navegador. Este “espacio en blanco” se usa solo para formatear el código de manera cómoda para el programador. Es una buena práctica que usen enter y tabulaciones para poder entender (de un primer vistazo) cómo está armada la estructura del sitio.

Existen etiquetas cerradas y abiertas:

Las cerradas encierran un contenido, por lo general texto. Las abiertas no encierran contenido, y sirven, entre otras cosas, para incluir elementos como imágenes, líneas, etc.

```
<p>Este es un párrafo con texto en su interior</p>  
<hr/>
```

En el ejemplo tenemos una etiqueta cerrada llamada Párrafo que engloba un texto y una etiqueta abierta para incluir una línea horizontal. El signo “/” se utiliza para las etiquetas de cierre; en las etiquetas cerradas se pone a continuación del signo “<”, en las abiertas se pone delante del signo “>”.

En HTML5, ya no es una obligación tener que poner el signo “ / “, p funcionará correctamente, lo mismo con
, <hr>, los meta tag.

Estructura básica de un documento:



Todos los documentos de HTML parten de la misma estructura básica que es necesaria para que funcione correctamente. Vamos a ir desglosando las etiquetas, para entender qué hace cada una.

Primero, debemos informar al navegador sobre el **tipo de documento** y la **versión de HTML** que estamos utilizando. Con la siguiente etiqueta establecemos que ésta será la versión 5 de HTML.

```
<!DOCTYPE html>
```

Debemos incluir las etiquetas de apertura y cierre. Dentro de ellas se desarrollará el contenido.

```
<html>
<!-- Aqui va el contenido del documento -->
</html>
```

Dentro de **<html>** existen dos partes principales del documento: **la cabeza (<head>)** y **el cuerpo (<body>)**.

En el sector **head** del documento, vamos a especificar información relacionada al navegador, importar archivos de estilos y, también, escribir el título de la página, etc.

En el **cuerpo** pondremos todo lo que el usuario verá reflejado en la **pantalla**. **La mayoría de nuestro trabajo ocurrirá en el body.**

```
<html>
  <head>
    <!-- Aqui va el contenido de head-->
  </head>
```

```
<body>
    <!-- Aqui va el contenido del cuerpo de nuestro documento -->
</body>
</html>
```

Ahora, para completar el esqueleto básico de una página web, sólo faltan dos elementos que van dentro de la etiqueta `<head>`: **el título y el tipo de caracteres utilizados.**

El título lo vamos a definir con una etiqueta `<title>` y vamos a poder verlo en la pestaña del navegador cuando entremos al sitio. Además lo vamos a ver en los resultados de búsqueda en sitios como Google, Yahoo, Bing.

```
<html>
  <head>
    <title>Mi sitio web</title>
  </head>

  <body>
    <!-- Aqui va el contenido del cuerpo de nuestro documento -->
  </body>
</html>
```

El set de caracteres que vamos a utilizar por nuestro alfabeto es el llamado **UTF-8** (que es, hoy en día, el más utilizado en Internet). Tenemos que identificarlo utilizando una etiqueta que se llama meta. La etiqueta meta contiene metadatos sobre nuestro sitio (como el autor, la descripción, etc.). En este caso, los caracteres son una propiedad. Otra novedad del elemento meta es que no puede tener contenido, todos los datos adicionales que necesitemos adjuntarle podrán ser escritos como propiedades. Importante: Esta es la primera etiqueta que encontramos que tiene etiqueta de apertura pero no una de cierre.

```
<html>

  <head>

    <meta charset="utf-8">

    <title>Mi sitio web</title>

  </head>

  <body>

    <!-- Aqui va el contenido del cuerpo de nuestro documento -->

  </body>

</html>
```

Los comentarios

Un último detalle. Si bien no son elementos html per sé, en los ejemplos de código encontramos unas etiquetas que tienen este formato:

```
<!-- Soy un comentario HTML -->
```

Esta estructura se repetirá en todos nuestros documentos **".html"**. Un detalle importante a tener en cuenta, es el nombre que le damos a nuestro documento principal. **Por convención**, se lo suele nombrar **index.html**, y es conveniente mantener esta convención para que más adelante, cuando subamos nuestro sitio a Internet, todo funcione como debe ser.

Tipos de etiqueta Grupo General

Todas las etiquetas que van dentro del **<body></body>** se dividen en dos grupos:

- **Elementos de bloque:** Son los que –sin ser modificados por CSS–, ocupan el 100% del ancho de su contenedor y se mostrarán uno abajo del otro.
- **Elementos de línea:** Solo ocupan el ancho que diga su contenido y se verán uno al lado del otro.

Estructura del cuerpo

Conceptos clave relacionados con la semántica en HTML:

Uso adecuado de elementos: HTML proporciona una amplia variedad de elementos para etiquetar diferentes partes del contenido. Es importante utilizar los elementos apropiados según su significado y función.

Jerarquía de contenido: La estructura del contenido debe reflejar su jerarquía lógica. Los elementos de nivel superior, como `<header>`, `<main>` y `<footer>`, deben utilizarse para definir secciones principales de una página, mientras que los elementos de nivel inferior, como `<section>`, `<article>` y `<div>`, se utilizan para dividir y organizar el contenido dentro de esas secciones.

No desarrollar un sitio de manera semántica tiene sus repercusiones en SEO y Accesibilidad por lo cual desarrollarlo bien es nuestro deber como desarrolladores y desarrolladoras para no solo lograr que nuestro sitio esté mejor posicionado, sino que sea accesible a todas las personas sin importar si tiene o no una discapacidad.

Etiquetas semánticas más comunes:

`<header>`: La cabecera del sitio. Contiene por lo general la barra de navegación.

`<nav>`: Barra de navegación. Puede estar en múltiples lugares dependiendo el diseño de la página.

`<main>`: Especifica el contenido principal de la página. Contiene contenido que resulta único para la página y no se replica en otras páginas (como el section, header, footer, etc). Solo se debe usar una sola etiqueta main en un mismo archivo. Una etiqueta main acepta múltiples sections.

`<section>`: Agrupa secciones dentro de una web. Todos los elementos dentro de la sección están relacionados. Pueden existir múltiples etiquetas section en un mismo archivo.

`<aside>`: Contiene información relacionada con el contenido principal, pero no es el contenido en sí mismo.

Agrega información como por ejemplo links relacionados, información del autor, publicidades relacionadas, etc.

<article>: Contiene información individual. Esta etiqueta se piensa como un bloque que funciona en sí mismo y podríamos integrarlo a cualquier web. Tiene un contexto propio. Algunos ejemplos son posts de un blog, noticias, e información que funciona en sí misma.

<footer>: El pie de página del sitio. Contiene por lo general información de contacto, el mapa del sitio y una barra de navegación del sitio entero.

<address>: Contiene información de contacto relacionado con la etiqueta que lo contenga.

<figure>: Se utiliza para representar contenido independiente, como imágenes, gráficos, ilustraciones, etc., junto con una leyenda utilizando la etiqueta **<figcaption>**.

<figcaption>: Define la leyenda o descripción de una figura definida con la etiqueta **<figure>**.

<time>: Se utiliza para representar fechas y horarios. Puede tener atributos adicionales como **datetime** para proporcionar información en un formato específico.

A la hora de armar un sitio web (siempre existe la excepción a la regla) pero deberían estructurarse de una manera similar:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Sitio Web</title>
</head>
<body>
```

```
<header>

  <nav>

    <!-- Barra de navegación -->

  </nav>

</header>

<main>

  <section>

    <h1>Título de la sección principal</h1>

    <!-- Contenido de la sección principal -->

  </section>

  <section>

    <h2>Subtítulo de otra sección</h2>

    <!-- Contenido de otra sección -->

  </section>

</main>

<aside>

  <!-- Contenido adicional relacionado -->

</aside>

<footer>

  <!-- Pie de página -->

</footer>

</body>

</html>
```

Beneficios del HTML Semántico

Al escribir HTML semántico obtenemos varios beneficios:

1. Accesibilidad:

- Aseguramos que personas que tienen distintas discapacidades (ya sea visual, auditiva, motriz, etc.) puedan navegar nuestro sitio web sin problemas. Además, también beneficia a aquellos que utilizan teclados u otros dispositivos de asistencia para navegar por el sitio.

2. Código legible, estructurado y mantenible.

- El uso de etiquetas semánticas y una estructura clara en el HTML hace que el código sea más fácil de leer y entender tanto para los desarrolladores como para otros miembros del equipo. Esto facilita el mantenimiento y las futuras actualizaciones del sitio, ya que es más sencillo localizar y modificar el contenido y la estructura.

3. SEO:

- Si nuestro sitio es accesible y está bien desarrollado, es más probable que nuestro sitio quede mejor posicionado en los resultados de los buscadores (Google, por ejemplo).

Encabezados

Los títulos en HTML son distintos a la etiqueta <title> de la que hablamos antes, porque title es el título del sitio web y **los encabezados son títulos dentro del contenido de nuestro sitio**. Estos últimos son definidos con la **letra h (heading, en inglés)** e identificados por números del **1 al 6**. El <h1> es el **encabezado más importante** y grande del sitio, por el contrario, el <h6> es el **menos importante** y más pequeño. Estos niveles de encabezados no representan un texto en negrita más grande o más chico, detrás de estos “niveles” existe una jerarquía. No debes tener un <h3> si antes no hubo un <h2>. Además están relacionados: Cada <h3> debe ser un sub-nivel de contenido del <h2> inmediato-anterior. Y si dos títulos son igual de importantes, tienen el mismo nivel de encabezado. Y si el título no es igual al anterior (ni tampoco es un sub-nivel) se volverá al nivel de encabezado anterior.

```
<h1> REINO ANIMAL </h1>
<h2> Vertebrados </h2>
  <h3> Mamíferos </h3>
    <h4> Bipedos </h4>
    <h4> Cuadrúpedos </h4>
  <h3> Aves </h3>
    <h4> Voladoras </h4>
    <h4> No voladoras </h4>
  <h3> Reptiles </h3>
```

```

<h3> Peces </h3>
<h3> Anfibios </h3>
<h2> Invertebrados </h2>
<h3> Insectos </h3>
  <h4> Voladores </h4>
  <h4> No Voladores </h4>

```

Una aclaración importante es que el **<h1>** sólo debe ser usado una vez en cada documento HTML. Los demás (del **<h2>** al **<h6>**) pueden repetirse las veces que sea necesario. También debemos recordar que no debemos elegir cuál usar según su tamaño sino por la relevancia del título en sí. El tamaño puede cambiar después.

Los párrafos

Una vez que tenemos el título entonces podemos agregar texto. Eso lo hacemos con el elemento párrafo **<p> (de bloque)**. Un párrafo es un texto formado por una o más oraciones, pero sólo debe encerrar texto. Se inserta con la etiqueta **<p></p>**. Dentro de un párrafo se pueden insertar otros elementos de texto (negritas, itálicas, acrónimos, abreviaturas, vínculos).

No puede haber un párrafo dentro de otro párrafo. Esto no existe en la literatura, y mucho menos en la estructura de un documento HTML.

```

<p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Quod ipsa pariatur sunt asperiores
rerum voluptates beatae quos quia voluptate in sint dolorum consectetur natus neque minima
temporibus accusantium, autem vero!</p>

```

Este tipo de texto falso se llama Lorem Ipsum por las palabras con las que comienza, y es utilizado en diseño desde hace décadas, cuando se necesita llenar un espacio de texto durante el diseño o desarrollo pero aún no se tiene el contenido final. En Internet pueden encontrarse muchísimos generadores de este tipo de texto, algunos de ellos con distintas temáticas.

Destacando contenido importante

En ocasiones vamos a necesitar destacar nuestro contenido, marcarlo como importante o darle énfasis. Para eso contamos con varias etiquetas específicas, dependiendo del resultado que queramos lograr.

¿Texto destacado, o sólo en negrita?

**
 (de línea):** Inserta un salto de línea simple dentro de un párrafo. En HTML, un salto de línea en el código fuente no tiene efecto visual, por lo que se utiliza esta etiqueta para forzar un salto. Su nombre proviene de *break* y su sintaxis correcta puede escribirse como **
** o **
** (ambas válidas en HTML5). Se recomienda emplearla solo para separar oraciones dentro de un mismo párrafo, no para dividir bloques de texto.

** (de línea):** Define texto visualmente destacado en **negrita**, sin añadir relevancia semántica. Se utiliza para resaltar palabras o frases sin indicar importancia o énfasis adicional.

** (de línea):** Indica un texto de **importancia semántica o relevancia** dentro del contenido. Los navegadores suelen representarlo en negrita, pero su función principal es semántica: comunica jerarquía o peso informativo.

<i> (de línea): Define texto en cursiva por razones estilísticas o convencionales, como nombres técnicos, títulos de obras o términos extranjeros. No implica énfasis semántico.

** (de línea):** Señala **énfasis semántico** dentro del texto. Al ser leído, puede generar un cambio de entonación o significado, diferenciándose del uso meramente estilístico de **<i>**.

Listas y listas anidadas

Las listas son un elemento muy común y muy útil para presentar información en una página web. A continuación veamos los tipos de listas que existen, para qué sirven y cómo crearlas.

Las listas ordenadas

Usamos las listas ordenadas cuando necesitamos mostrar información para la cual el orden es importante. Por ejemplo, un ranking de éxitos musicales o un listado de posiciones de un torneo de fútbol. En estos casos cambiaría totalmente el sentido del listado si cambiáramos puestos de lugar. En la vista del navegador, este tipo de listado es acompañado de números mostrando el orden que elegimos.

Para crear la lista usamos la etiqueta `<ol>` y para separar cada ítem dentro de ella utilizamos las etiquetas `<li>`:

```
<h2>Países más poblados del mundo</h2>
<ol>
  <li>China - 1.448.471.404 habitantes</li>
  <li>India - 1.406.631.781 habitantes</li>
  <li>Estados Unidos - 334.805.268 habitantes</li>
</ol>
```

Las listas desordenadas son iguales a las ordenadas, excepto que las usamos para listar ítems en las que el orden no es importante. Por ejemplo, una lista de supermercado. Se encierran en una etiqueta `<ul>`:

```
<h2>Lista de compras</h2>
<ul>
  <li>Manzanas</li>
  <li>Leche</li>
  <li>Queso</li>
</ul>
```

Pongamos en práctica lo visto:

Consigna #1:

A partir de lo visto hasta acá, te invitamos a que de esta lista desordenada, puedas realizar los cambios necesarios dentro del html para que esta lista se convierta en una lista ordenada:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title></title>
</head>
<body>
  <ul>
```

```

<li>CPU</li>
<li>MEMORIA</li>
<li>MOTHERBOARD</li>
<li>MONITOR</li>
</ul>
</body>
</html>

```

Consigna #2:

Dentro de esta estructura HTML, creá una cabecera h1, una lista ordenada o desordenada y un párrafo y recordá completar la etiqueta title.

Cada elemento debe tener contenido dentro, imagina una lista para el super, las pelis que más te gustan, etc y contanos algo sobre el contenido que está en la lista.

Ejemplo: Lista de todas las películas de Star Wars y contó sobre su dirección y producción

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title></title>
</head>
<body>
</body>
</html>

```

😊 Podrás encontrar la solución de estos ejercicios al final de este manual, en el apartado “Resolución desafío”. Te recomendamos que la mires una vez que hayas finalizado la actividad.

Enlaces o hipervínculos

Cada página, foto, video, audio, pdf, en fin, cada recurso que encontramos en Internet, tiene un nombre que lo identifica de manera única. Se trata de su **URL, Identificador Único de Recursos** (del inglés Uniform Resource Locator). Las URL tienen dos funciones. Por un lado, como ya dijimos, identifican de forma única a

cada recurso en Internet. Además, también le brindan a los navegadores la información necesaria para poder llegar hasta ese recurso: son la ruta del recurso.

Por otro lado, las **URL** también proporcionan información necesaria para que los navegadores puedan encontrar y acceder al recurso en cuestión. La **URL** contiene la ruta del recurso, es decir, la dirección precisa en la que se encuentra almacenado o ubicado. Esta ruta puede incluir el protocolo de comunicación utilizado (como **HTTP** o **HTTPS**), el dominio del sitio web (como "google.com"), directorios o carpetas específicos dentro del sitio, y el nombre del archivo o recurso en sí.

Al ingresar una URL en el navegador web, este utiliza la información proporcionada en la URL para establecer una conexión con el servidor donde se encuentra alojado el recurso deseado. El **servidor** envía luego los **datos** del recurso de vuelta **al navegador**, que los interpreta y muestra al usuario en forma de página web, imagen, vídeo u otro tipo de contenido.

<https://www.alkemy.org/faq-items/que-es-el-screening/>

Las partes que componen la URL son:

- **Protocolo (https://)**: el mecanismo que debe utilizar el navegador para acceder a ese recurso. Todas las **páginas web seguras** utilizan https://.
- **Host (www.alkemy.org)**: es el servidor donde se encuentra guardada la página a la que queremos acceder. Los navegadores son capaces de **obtener la dirección de cada host** a partir de su nombre.
- **Ruta (/faq-items/que-es-el-screening/)**: Es el camino de carpetas que se debe seguir, una vez que llegamos al servidor, para localizar el recurso específico que queremos acceder.

Rutas Absolutas

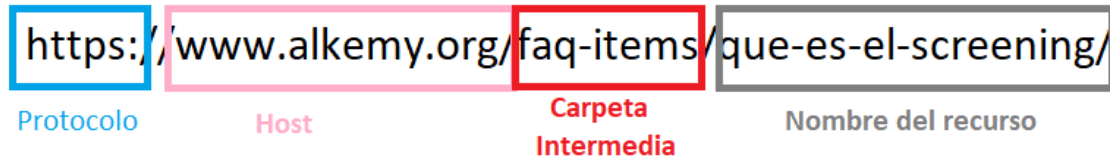
Las **rutas absolutas** incluyen **todas las partes** de la **URL (protocolo, servidor y ruta)** por lo que no se necesita más información para obtener el recurso enlazado. Se utilizan cuando queremos enlazar algún recurso que se encuentra hospedado en otro sitio web.

En formato HTML se escribiría de la siguiente forma:

```
<a href="https://www.alkemy.org/faq-items/que-es-el-screening/">¿Qué es el screening?</a>
```

Vemos que la ruta absoluta coincide completamente con la URL del recurso.

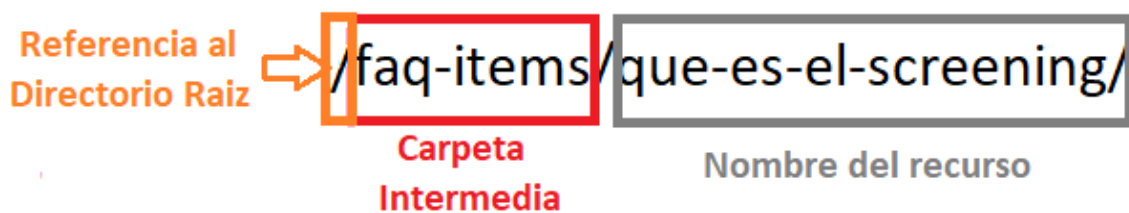
La "anatomía" de la ruta absoluta es ésta:



Rutas Relativas

Las **rutas relativas** son las rutas más utilizadas en Web, y reciben ese nombre porque indican el **camino para encontrar un elemento**, pero basándonos en el **directorio (carpeta)** en donde nos encontramos posicionados. **Omiten la parte del protocolo, nombre del host e incluso parte o toda la ruta del recurso** enlazado para hacerlas más breves. Como se trata de rutas incompletas, necesitamos información adicional para llegar al recurso enlazado.

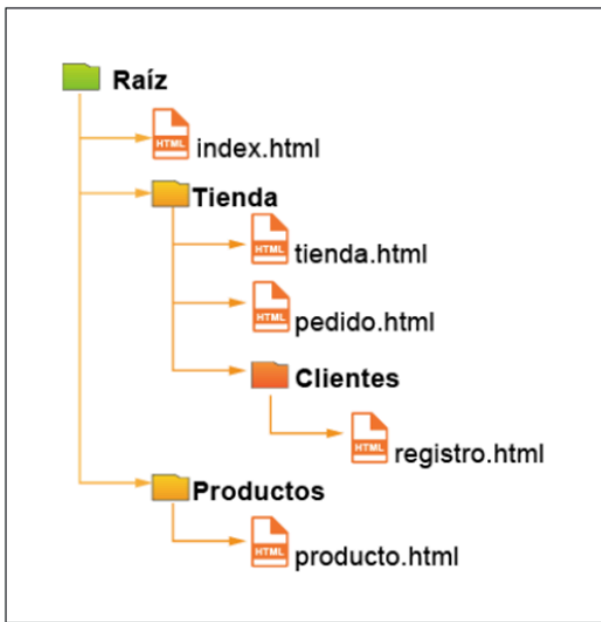
Una ruta relativa se ve así:



La principal ventaja que ofrece este tipo de rutas es que **facilita mucho el mantenimiento de una web**, permitiendo mover el contenido de un host a otro sin tener que hacer ningún cambio en las rutas. En el caso de las absolutas, cambiar de host conlleva tener que modificar todas las rutas absolutas para indicarle el nombre del nuevo host.

Cómo usar rutas relativas:

Dependiendo de hacia dónde queramos enlazar, vamos a tener que escribir nuestra ruta relativa de manera distinta. Para que se entienda mejor, vamos a crear un "árbol de directorios", y nos vamos a referir a sus componentes.



Paso a paso:

1. Enlace hacia un archivo que está en el mismo directorio Si estamos dentro de la carpeta "tienda" la ruta al pedido es directamente "pedido.html".
2. Crear un enlace hacia un archivo que está en una subcarpeta del mismo nivel Si estamos dentro de la carpeta "tienda" y queremos acceder al archivo "registro.html" que se encuentra en una carpeta diferente, la ruta relativa sería "clientes/registro.html". Utilizamos la barra diagonal (/) para separar las carpetas.
3. Enlazar hacia un archivo que se encuentra en un nivel superior Si estamos dentro de la carpeta "tienda" y queremos acceder al "index.html" la ruta sería "../index.html". El uso de dos puntos y una barra oblicua (../) nos permite subir un nivel en la jerarquía de carpetas. Si tuviéramos que subir varios niveles, se podría utilizar la estructura "../" tantas veces como nos hiciera falta, por ejemplo subir dos niveles sería "../../index.html".
4. Enlazar hacia un archivo que se encuentra en una subcarpeta distinta Si estamos dentro de la carpeta "tienda" y queremos acceder a producto.html, lo que hacemos es utilizar "../" para subir un nivel, para después hacer uso de "productos/" para entrar dentro de la carpeta donde se encuentra el archivo "producto.html". La ruta relativa sería: "../productos/producto.html" Rutas relativas a la raíz del sitio

- Indica la ruta completa desde la raíz de un sitio web hasta el archivo que queremos enlazar. Las rutas relativas de este tipo siempre comenzarán con una barra (/) que hace referencia al directorio raíz del sitio. Por ejemplo, `"/tienda/clientes/registro.html"` es un enlace relativo a la raíz del sitio, ya que como podemos ver, empieza con una barra y recorre todas las carpetas y subcarpetas del árbol hasta llegar al recurso en cuestión.

Navegación a documentos externos

Atributo href:

Para que el vínculo funcione es necesario informarle a dónde queremos que nos lleve al hacerle clic. Esto se hace con el atributo href. Si se trata de un enlace a un sitio web externo debemos escribir la dirección completa con la cual deseamos que nos comunique.

Por ejemplo:

```
<a href="https://www.google.com">Google</a>
```

Si queremos ir a otro documento dentro de la misma carpeta de nuestro documento principal, alcanza con poner el nombre del documento y su extensión:

```
<a href="pagina2.html">Página 2</a>
```

Atributo target:

Los links poseen además otro atributo muy útil llamado target. Este atributo nos ayuda a definir (mediante diferentes valores que podemos darle) en dónde queremos que se abra el documento vinculado. Por defecto, el valor que posee es `"_self"`, esto significa que los vínculos se van a abrir siempre en la misma pestaña en la que nos encontramos:

```
<a href="pagina2.html" "target="_self">Página 2</a>
```

Si queremos que el documento o el sitio externo se abra en una nueva pestaña, podemos utilizar "_blank":

```
<a href="https://www.google.com" target="_blank">Google</a>
```

Imágenes

Los documentos html no tienen por qué basarse solamente en contenido de texto, tenemos también opciones de elementos multimedia como, por ejemplo, **imágenes, video y audio**.

Existen **diferentes tipos de archivos de imágenes** que cumplen el mismo objetivo: mostrar una foto o imagen.

Los **archivos JPG (o JPEG), PNG y GIF** suelen ser los más habituales en diseño web y los que vas a ver más seguido, pero existen otros formatos, como el **SVG**, y cada uno tiene sus fortalezas y debilidades.

Estos son los tipos de formatos (extensiones) que podemos encontrar para imágenes:



Agregar imágenes externas

Para agregar **imágenes externas** tenemos que copiar el **hipervínculo** directo de la imagen. Si estoy, por ejemplo, tratando de obtenerlas desde Google Images, tengo que hacer clic derecho en la imagen y seleccionar "Abrir imagen en nueva pestaña" para poder copiar luego esa dirección.

El elemento que utilizamos para imágenes es el **** que no requiere etiqueta de cierre. Indicamos la dirección a la imagen en el atributo **src**.

```

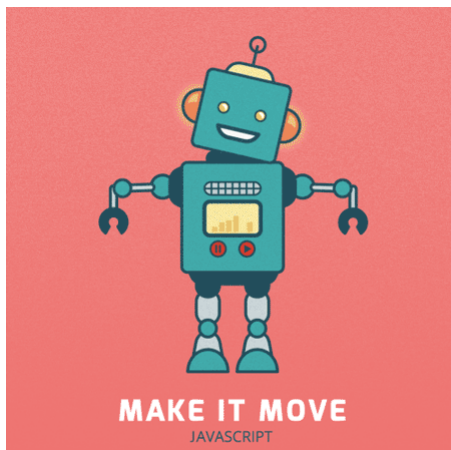
```

Los atributos de la etiqueta ****:

alt = Es el valor que hay que agregar es una descripción de la imagen.

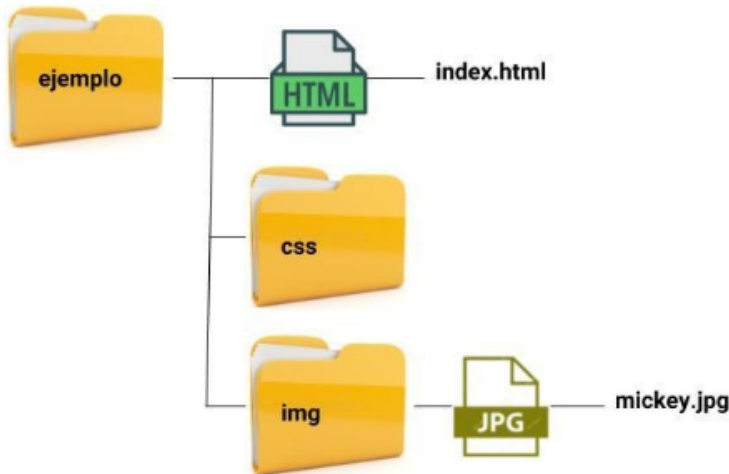
src = Se utiliza para agregar la ubicación de la imagen.

Resultado:



Agregar imágenes internas

Si la imagen se encuentra en la misma carpeta que tu archivo HTML, se puede utilizar simplemente el nombre de archivo (incluyendo la extensión) en el atributo **src** de la etiqueta ****.



```

```

Incluir elementos externos 🖥️

El elemento **<iframe>** se utiliza para insertar y mostrar contenido interactivo de otros sitios web dentro de una página HTML. Podemos utilizarlo para **incrustar videos, mapas, aplicaciones** o cualquier otro tipo de recurso web dentro de tu propia página.

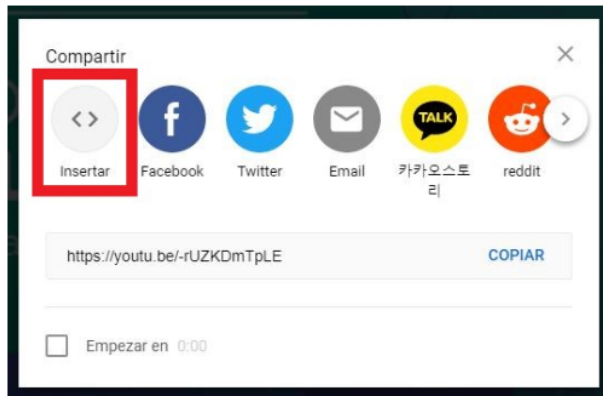
El **<iframe>** crea un espacio dentro de tu documento HTML en el que se carga el **contenido externo de forma independiente**. Esto significa que el **contenido incrustado** tiene sus propios estilos, funcionalidades y comportamiento. Incluso si el contenido dentro del **<iframe>** contiene enlaces o scripts, se mostrarán y ejecutarán dentro de ese espacio del **<iframe>**.

Es una forma común de integrar contenido de otros sitios web en tu propia página, como mostrar videos de YouTube, mapas de Google Maps, widgets de redes sociales o aplicaciones externas. Esto te permite enriquecer tu página con contenido interactivo y funcionalidades adicionales provenientes de diversas fuentes en línea.

Si queremos incrustar un video de YouTube en nuestra página HTML utilizando un **<iframe>**, hay que seguir estos pasos:

1. Ir a la página del video de YouTube que desees incrustar.
2. Haz clic en el botón "Compartir" debajo del video.
3. Selecciona la opción "Incrustar" en el menú desplegable.

4. A continuación, se mostrará el código de incrustación del video. Copia el código proporcionado y pégalo en tu página HTML donde desees que aparezca el video.



En este caso, estamos embebiendo un video de YouTube.

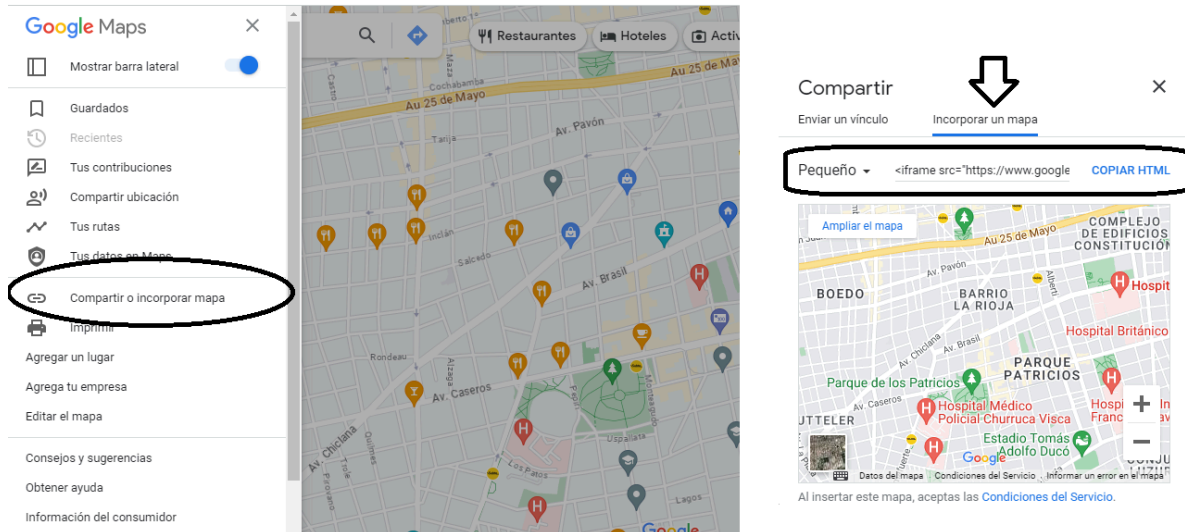
```
<iframe width="560" height="315" src="https://www.youtube.com/embed/-rUZKDmTpLE" frameborder="0"
allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
```

Utilizando un <iframe> para embeber mapas 🗺️

Ya conocimos el elemento <iframe>, y lo usamos para insertar videos de YouTube en nuestro sitio. De manera similar, podemos utilizarlo para agregar mapas. Al igual que con los videos, el proceso es sencillo. Aquí están los pasos necesarios:

1. Lo primero que debemos hacer es acceder a Google Maps. Elegimos la dirección que queremos mostrar en el mapa y ajustamos el zoom.
2. En la esquina superior izquierda hacemos clic sobre el menú hamburguesa (las tres líneas horizontales paralelas). Ahí seleccionamos la opción "Compartir o insertar el mapa".
3. Se abrirá una ventana en la que aparece el enlace para compartir. Vamos a elegir la opción "Insertar mapa". Ahí podemos definir el tamaño que mejor se adapte a nuestro sitio (pequeño, mediano, grande o tamaño personalizado).

- Una vez elegido el tamaño, copiamos el código del `<iframe>`. Nuevamente, el último paso es pegarlo en nuestro documento HTML, y probarlo en el navegador.



Tablas

La etiqueta `<table>` se utiliza para crear la tabla en sí. Las tablas son un **conjunto de celdas organizadas**, dentro de las cuales es posible alojar distintos contenidos. HTML dispone de una gran variedad de etiquetas y atributos para crear tablas. **Sirven para representar información tabulada, en filas y columnas.**

Algunos puntos clave sobre las tablas en HTML son:

Estructura de filas y columnas:

Las tablas se componen de **filas** (`<tr>`) y **columnas** (`<th>` y `<td>`). Las filas representan conjuntos de celdas que se encuentran en la misma línea horizontal, mientras que las columnas se definen mediante celdas en cada fila.

Etiquetas `<th>` y `<td>`:

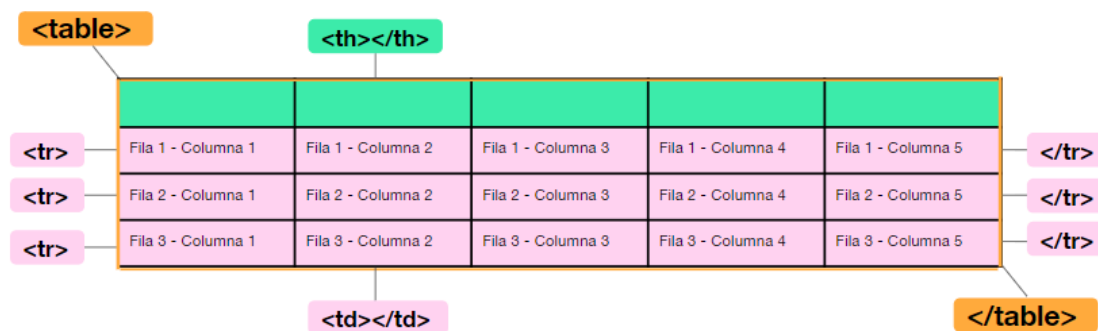
La **etiqueta** `<th>` se utiliza para **definir celdas de encabezado**, que generalmente se encuentran en la primera fila de la tabla y describen el contenido de las columnas. La **etiqueta** `<td>` se utiliza para **las celdas de datos**, que contienen el contenido real de la tabla.

Atributos adicionales:

HTML proporciona varios atributos para personalizar las tablas, como el **ancho de la tabla (width)**, el **espaciado entre las celdas (cellspacing)**, el **relleno interno de las celdas (cellpadding)**, el **alineamiento de las celdas (align)**, entre otros.

Estilización con CSS:

Las tablas pueden ser estilizadas y personalizadas utilizando CSS. Esto permite cambiar el aspecto visual de la tabla, como los colores, fuentes, bordes y estilos de las celdas, entre otros.



La estructura de de etiquetas sería de esta manera:

```
<table>
<tr><!-- inicio de fila-->
  <td>Fila 1 - Columna 1</td>
  <td>Fila 1 - Columna 2</td>
  <td>Fila 1 - Columna 3</td>
</tr><!-- cierre de fila -->
<tr><!-- inicio de otra fila-->
  <td>Fila 2 - Columna 1</td>
  <td>Fila 2 - Columna 2</td>
  <td>Fila 2 - Columna 3</td>
</tr><!-- cierre de la segunda fila -->
</table>
```

Y el resultado obtenido sería el siguiente:

```
Fila 1 - Columna 1 Fila 1 - Columna 2 Fila 1 - Columna 3
Fila 2 - Columna 1 Fila 2 - Columna 2 Fila 2 - Columna 3
```

Para que nuestra tabla empiece a tomar estilo no nos olvidemos que la tabla acepta 3 atributos de “diseño”:

- **Border:** bordes de la tabla.
- **Cellpadding:** especifica el espacio, en píxeles, entre la pared de la celda y su contenido.
- **Cellspacing:** indica la distancia entre las celdas y el margen exterior de la tabla

Formularios

Los **formularios en HTML** son elementos que permiten recopilar y enviar datos ingresados por los usuarios. Son etiquetas donde el usuario ingresará o seleccionará valores, que serán enviados a un archivo encargado de procesar la información.

Para insertar un formulario se usa la etiqueta **<form>**, que dentro lleva todos los controles que vayan al mismo destino. Un formulario requiere 3 atributos para funcionar:

- **Action:** documento que se encarga de recibir los datos y procesarlos.
- **Method:** la forma en que será enviada la información. Existen dos métodos de envío, que son GET y POST.
- **Enctype:** cómo se codificarán los contenidos.

```
<form action="/procesar-formulario" method="post">
</form>
```

action="" es el este atributo se indicará **cuál es el archivo que recibe y procesa los datos. Debe ser de un lenguaje de los llamados “del lado del servidor” (PHP / ASP / JSP)**. Si no se indica un valor, el Action será por defecto el mismo archivo donde está el formulario.

method="" es la forma en la que se recopilan y envían los datos. Existen dos métodos comunes en el HTML:

- **GET:** la información viajará por la barra de direcciones a continuación del nombre del archivo.
- **POST:** la información viajará junto a los encabezados del HTML (será “invisible”).

Si el method no se indica, por defecto será GET.

Cuando el valor del atributo method es post, el mismo es el [tipo MIME](#) del contenido, que es usado para enviar el formulario al servidor. Los posibles valores son:

- **application/x-www-form-urlencoded:** será el valor por defecto si un atributo no está especificado.
- **multipart/form-data:** usar este valor si se está usando el elemento input con el atributo type ajustado a "file".
- **text/plain** (HTML5)

Normalmente se utiliza para permitir el envío de archivos a través de un formulario.

Existen tres controles generales para el ingreso de texto:

- Cajas de texto de una sola línea (no acepta el uso de la tecla Enter).
- Cajas para el ingreso de contraseñas (el contenido no será visible).
- Cajas para contenido multilínea. Puede ser una o muchas líneas de texto.

Atributo **"name"**

Control de formulario: **<input>: Text, Email, Password.**

Control de formulario: **<textarea></textarea>**

Los botones disparan las acciones del formulario. Hay 3 tipos:

- El que envía los datos al archivo indicado como Action.
- El que vacía todo lo ingresado y resetea los campos.
- El que "no hace nada", pensado para usarse con Javascript.
-

Todos los botones son etiquetas **<input>**, con distintos tipos de "Type". El botón debe de estar dentro del **<form>** que afectará.

El atributo value representa **la etiqueta del botón <button>**, la cual es normalmente mostrada por los navegadores dentro de éste.

- **Input de tipo "submit":** envía el formulario.
- **Input de tipo "reset":** resetea el formulario.
- **Input de tipo "button":** no tiene acciones por defecto.

```
<form>
  <input type="submit" value="Enviar formulario"/>
  <input type="reset" value="Limpiar formulario"/>
  <input type="button" value="Sin acciones"/>
</form>
```

Los controles de selección solo se utilizan si queremos que el usuario no pueda ingresar libremente un texto, sino que el programador le da una lista predefinida. El dato que llega al elegir una opción se define desde el atributo “value”. Existen 3 grupos de controles de selección:

- **Botones de radio:** solo se puede elegir una opción.
- **Casillas de chequeo:** de toda la lista de opciones, el usuario puede optar por una, todas o ninguna opción.
- **Menú desplegable:** solo es posible seleccionar una opción.

Botones de radio:

```
<form>
  <div>hombre</div>
  <input type="radio" name="sexo" value="hombre" />
  <div>mujer</div>
  <input type="radio" name="sexo" value="mujer" />
</form>
```

Casillas de chequeo:

```
<form>
  <div>Acepta términos y condiciones</div>
  <input type="checkbox" name="acepta" value="1" />
</form>
```

La etiqueta <label> define formalmente a cada elemento de un formulario. Esta etiqueta es de mucha ayuda para generar un formulario accesible.

Su principal atributo es “for”, que va a referenciar a “label” con su elemento del formulario. El valor del atributo “for” debe ser igual al valor del atributo “id” o “name” del elemento.

```
<form>
  <label for="nombre_apellido">Nombre:</label>
  <input type="text" name="nombre_apellido" />
</form>
```

Para crear un menú desplegable, generalmente llamado **combo-box**, **selector** o **menú**. El cual, de toda la lista, se puede elegir una opción (aunque tiene un atributo que permite cambiarlo). Lo ideal es que sean al menos dos elementos distintos. Para esto se utiliza la etiqueta **<select>** y la etiqueta **<option>**

```
<form>
  <select name="talles">
    <option value="L">Large</option>
    <option value="M">Medium</option>
    <option value="S">Small</option>
  </select>
</form>
```

Las etiquetas **<fieldset>** y **<legend>** se utilizan en conjunto. La primera, tiene como objetivo crear grupos de elementos del formulario que posean un mismo propósito; mientras que la segunda, define formalmente el propósito del elemento fieldset. Se estructuran de la siguiente manera:

```
<form>
  <fieldset>
    <legend>Talle de remera</legend>
    <!-- Aquí irán los elementos de formulario -->
  </fieldset>
</form>
```

Utilizando las etiquetas mencionadas, nuestro formulario quedaría de la siguiente manera:

```
<form action="/procesar-formulario" method="post">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" placeholder="Ingrese su nombre" required>
  <br>
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" placeholder="Ingrese su email" required>
  <br>
  <label for="edad">Edad:</label>
  <input type="number" id="edad" name="edad" min="18" max="99" required>
  <br>
  <label for="pais">País:</label>
  <select id="pais" name="pais" required>
    <option value="">Seleccione su país</option>
    <option value="1">Argentina</option>
    <option value="2">Brasil</option>
    <option value="3">México</option>
  </select>
  <br>
  <label>Género:</label>
  <input type="radio" id="genero-m" name="genero" value="masculino" required>
  <label for="genero-m">Masculino</label>
  <input type="radio" id="genero-f" name="genero" value="femenino" required>
  <label for="genero-f">Femenino</label>
  <br>
  <button type="submit">Enviar</button>
  <button type="reset">Limpiar</button>
</form>
```

Nombre:

Email:

Edad:

País:

Género: ☐ Masculino ☐ Femenino

Elemento DIV

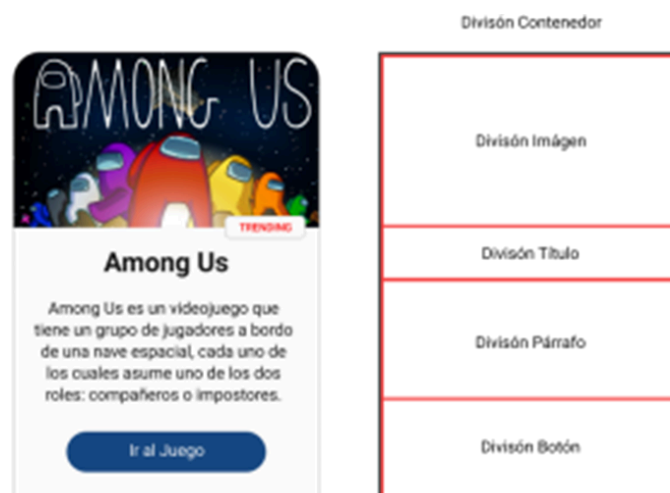
Mirada general y específica

Cuando estructuramos nuestro sitio ya sabemos que tenemos disponibles las etiquetas semánticas, sin embargo, pasa que muchas veces necesitamos distintos contenedores para poder armar estructuras más complejas.

Imaginemos un sitio desde una mirada genérica: tiene cabecera, contenido principal, tal vez contenido secundario y un footer, algo así:

Esto lo podemos resolver sin problemas gracias a las etiquetas semánticas que conocemos.

Pensemos ahora en ir un poco al detalle e imaginamos que la sección principal (el `<main>`) se ve así:

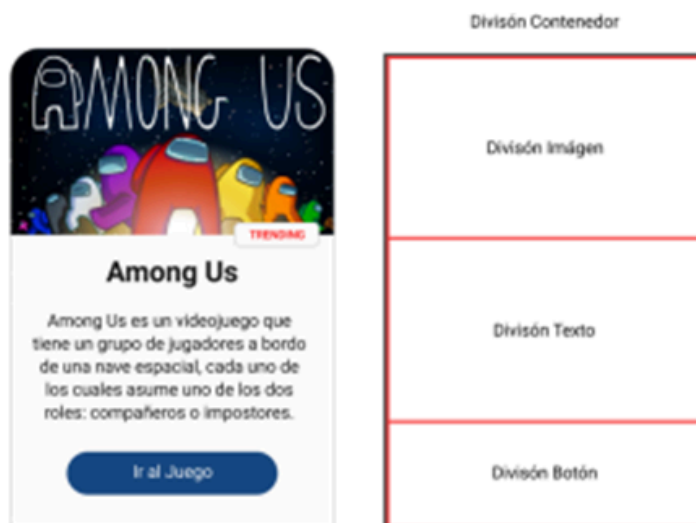


Si observamos, podemos ver que la etiqueta **<main>** contiene la información principal la cual esta dividida por **<section>** y en su interior hay tres “cards” y estas tienen un contenedor, una imagen, un título, texto y un botón. Podemos pensar en divisiones dentro de la card:

1. La división de la imagen
2. La división del título
3. La división de la información
4. La división del botón

Estas divisiones no son semánticas. Entonces, pensando a groso modo tenemos las etiquetas semánticas que nos permiten entender qué es una cabecera, contenido principal, secciones, pie de páginas, etc.

Pero cuando pensamos más en detalle, vemos que hay muchas divisiones más, que no necesariamente son semánticas, son divisiones que nos ayudan a estructurar nuestro sitio. Para esto, entra la etiqueta **<div>** (**etiqueta que NO es semántica**)



La etiqueta **<div>** sirve para crear secciones o agrupar contenido. Cuando vimos en detalle la sección del main descubrimos que se componía por cards y que estas tenían distintas divisiones. Si analizamos todo un poco más en detalle podríamos armar distintas maneras de agrupar el contenido de las cards.

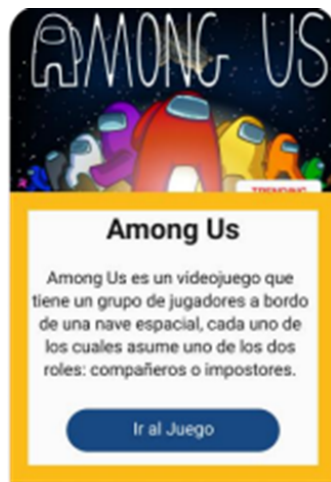
A la hora de agrupar contenido existen muchas maneras de verlo y es muy probable que todas sean válidas. Con el tiempo el desafío es entender que hay ciertos contenedores que solemos crear que se pueden evitar, pero eso es algo que vendrá con el día a día de desarrollo.

La importancia de agrupar contenido

La pregunta es... ¿Y para qué me sirve agrupar contenido? Y la respuesta a esto tiene que ver con tres cosas principales:

1. Crear código legible
2. Crear una estructura escalable que pueda cambiar con el tiempo
3. Generar secciones que tendrán estilos particulares

Respecto de las primeras dos, mientras mejor estructuramos los elementos html que creamos, más simple será mantenerlos y cambiarlos en el futuro. Respecto de la última, lo podemos analizar simplemente pensando en el padding de CSS. A continuación vemos en amarillo el padding de una de las divisiones de la card:



Viendo esta card, nos damos cuenta que hay un padding en común a todas las divisiones que no son la imagen, entonces podemos pensar en una estructura que tenga:

1. Un <div> contenedor de toda la card
2. Un <div> contenedor de la imagen
3. Un <div> contenedor de la información (este será el div que tenga el padding)
4. Si consideramos necesario, un div para el título y el párrafo (los grupo que contienen solo texto)
5. Un div contenedor del botón (que nos servirá para
6. centrar al botón).

Como podemos ver, la manera de dividir la card pensando en el padding, es distinta a las primeras propuestas de divisiones. Esta que recién mencionamos se vería algo así:



Características de un <div>

- Es una etiqueta de bloque, es decir que va a ocupar el 100% de la fila que ocupa.
-
- Si no tiene contenido, no se ve dado que no tiene alto (height)
-
- No tiene márgenes ni padding, si queremos que tengan debemos agregarlos al CSS.

Introducción del

El es la etiqueta **hermana del <div>**. En vez de agrupar todo tipo de contenido, lo vamos a pensar como el contenedor de etiquetas o elementos de línea.

Para entenderlo pensemos en este ejemplo concreto: imaginemos un texto que tiene una frase de otro color

sumamente importante

¿Cómo lograríamos esto?

Cómo estamos cambiando el color, sabemos que vamos a tener que usar CSS en algún momento, y lo que hacemos en “envolver” en un `<span>` el texto que queremos resaltar, y le cambiamos los estilos mediante CSS.

```
<p>Es <span> sumamente importante </span> que complete su DNI sin espacios </p>
```

La **diferencia radical** de estas etiquetas tan similares, es que el `<div>` **acepta dentro suyo cualquier etiqueta mientras que el `<span>` solo acepta etiquetas o elementos de línea** (por ejemplo una etiqueta `<a>` o simplemente texto).

La otra gran diferencia es que el `<div>` **es una etiqueta de bloque que ocupará el 100%** de la fila que ocupa, empujando las etiquetas siguientes hacia abajo mientras que el `<span>` **es de línea** y ocupa solamente lo que su contenido ocupe, sin empujar ningún elemento hacia abajo.



Cierre

En esta unidad, hemos explorado los fundamentos del lenguaje de marcado HTML, una de las piedras angulares de la creación de páginas web. Hemos aprendido cómo estructurar el contenido de manera jerárquica utilizando elementos como encabezados, párrafos, listas y enlaces. Además, hemos descubierto cómo agregar imágenes y contenido multimedia a nuestras páginas para enriquecer la experiencia del usuario.

Comprender el funcionamiento de las URL y saber cómo utilizar tanto rutas absolutas como relativas en HTML es esencial para desarrollar sitios web eficientes y que brinden una experiencia de usuario óptima. La correcta gestión de las rutas en el diseño y desarrollo de páginas web nos permite acceder y compartir recursos de manera efectiva en Internet, facilitando la navegación y la interacción con el contenido digital en la actualidad.

Entendimos que el elemento `div` y `span`, ambos elementos son fundamentales para la maquetación y manipulación de contenido en HTML y son ampliamente utilizados en combinación con CSS y JavaScript para crear diseños más complejos y dinámicos. Su flexibilidad y capacidad para agrupar y estilizar elementos de manera selectiva hacen que sean herramientas indispensables en el desarrollo web moderno.

Referencias

Mozilla. (s.f.). HTML5 – Glosario. MDN Web Docs.

<https://developer.mozilla.org/es/docs/Glossary/HTML5>

¡Muchas gracias!

Nos vemos en la próxima lección

